



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

UVT Asist: asistent inteligent pentru accesul la
informațiile oficiale ale UVT

COORDONATOR:

Asist. Drd. Theodor Radu Grumeza

ABSOLVENT:

Pavel Cătălin

TIMIȘOARA

2026

UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

UVT Asist: asistent inteligent pentru accesul la
informațiile oficiale ale UVT

COORDONATOR:

Asist. Drd. Theodor Radu Grumeza

ABSOLVENT:

Pavel Cătălin

TIMIȘOARA

2026

Abstract

This bachelor thesis presents the design and implementation of **UVT Asist**, a local retrieval-augmented assistant dedicated to students of the West University of Timișoara. The application is delivered as a Chrome extension popup and is supported by a Flask backend that indexes official university and faculty pages, retrieves relevant evidence, and generates concise answers grounded in official sources.

The proposed system follows a local-index-first RAG architecture. UVT pages are crawled, cleaned, split into chunks, embedded locally with Ollama, stored in Qdrant, and searched using semantic retrieval combined with deterministic reranking. The assistant uses metadata filters such as faculty and page type, detects common student intents, corrects frequent Romanian query typos, and exposes official source links for every answer. When evidence is weak, the system avoids unsupported claims and communicates the limitation explicitly.

The thesis discusses the theoretical basis of conversational assistants, information retrieval, vector search, large language models and retrieval-augmented generation. It then maps these concepts to the implemented prototype, emphasizing transparency, local execution, source verification, and practical usability for students searching for schedules, admissions information, scholarships, regulations, contacts or other administrative resources.

Rezumat

Lucrarea prezintă proiectarea și implementarea aplicației **UVT Asist**, un asistent inteligent local, orientat către studenții Universității de Vest din Timișoara. Scopul sistemului este reducerea timpului necesar pentru găsirea informațiilor oficiale publicate pe site-ul universității și pe site-urile facultăților. Produsul final este o extensie Google Chrome, iar procesarea întrebărilor este realizată de un backend Flask care construiește contextul relevant și returnează răspunsuri scurte, însoțite de surse oficiale.

Soluția folosește o arhitectură de tip Retrieval-Augmented Generation. Paginile oficiale sunt descoperite, descărcate, curățate, împărțite în fragmente, transformate în reprezentări vectoriale cu Ollama și stocate în Qdrant. La interogare, sistemul combină căutarea semantică, filtrele pe facultate și tip de pagină, reranking-ul determinist și generarea locală a răspunsului. Pentru întrebările de navigare sau pentru cazurile cu dovezi insuficiente, backend-ul poate evita modelul generativ și poate folosi răspunsuri deterministe sau mesaje explicite de incertitudine.

Lucrarea tratează fundamentele teoretice ale asistenților conversaționali, ale sistemelor de regăsire a informației, ale modelelor de limbaj și ale căutării vectoriale, apoi le conectează cu deciziile tehnice din proiectul UVT Asist. Accentul este pus pe utilizarea surselor oficiale, pe transparența răspunsurilor, pe rularea locală a componentelor AI și pe limitarea riscului de generare a unor informații neverificate.

Cuprins

1	Introducere	7
1.1	Contextul lucrării	7
1.2	Motivație	7
1.3	Scopul și obiectivele lucrării	8
1.4	Contribuții	8
1.5	Structura lucrării	9
2	Fundamente teoretice	10
2.1	Regăsirea informației	10
2.2	Modele de limbaj și limitele lor	10
2.3	Embeddings și căutare vectorială	11
2.4	Retrieval-Augmented Generation	11
2.5	Extensii de browser ca interfață	12
2.6	Rulare locală și confidențialitate	12
2.7	Calitatea surselor și prospețimea informației	13
2.8	Degradare controlată	13
3	Analiza problemei și cerințe	15
3.1	Particularitățile ecosistemului UVT	15
3.2	Cerințe funcționale	15
3.3	Cerințe nefuncționale	16
3.4	Categorii de întrebări	16
3.5	Cazuri de utilizare reprezentative	17
3.6	Riscuri informaționale	17
3.7	Criterii de acceptare	18
4	Arhitectura sistemului UVT Asist	19
4.1	Viziune generală	19
4.2	Principii de proiectare	20
4.3	Modelul datelor	20
4.4	Fluxul de indexare	21

4.5	Fluxul de interogare	22
4.6	Controlul generării	23
4.7	Verificare live și cache	23
5	Componente și funcționalități	25
5.1	Structura proiectului	25
5.2	Extensia Chrome	26
5.3	Backend-ul Flask	26
5.4	Indexarea și extragerea conținutului	27
5.5	Configurația facultăților	28
5.6	Retriever-ul	28
5.7	Integrarea Ollama și Qdrant	28
5.8	Scorul de încredere	29
5.9	Răspunsuri, surse și feedback	29
6	Validare, limite și direcții viitoare	30
6.1	Strategia de validare	30
6.2	Testarea automată	30
6.3	Evaluarea scenariilor manuale	31
6.4	Metrici calitative	31
6.5	Limite	32
6.6	Amenințări la validitate	32
6.7	Considerente de securitate și confidențialitate	33
6.8	Direcții viitoare	33
7	Concluzii	34
8	Plan de activitate	35
	Bibliografie	36

Capitolul 1

Introducere

1.1 Contextul lucrării

Digitalizarea serviciilor universitare a schimbat modul în care studenții interacționează cu instituția. Informații precum orare, calendare administrative, regulamente, metodologii de burse, proceduri de admitere, contacte, formulare sau anunțuri sunt publicate în mod curent pe site-uri web. Această disponibilitate nu înseamnă însă automat acces rapid. Studentul trebuie să știe unde să caute, să identifice pagina corectă, să interpreteze termenii folosiți de instituție și să distingă între o pagină generală, o știre veche și o metodologie încă relevantă.

În cazul Universității de Vest din Timișoara, informația este distribuită între site-ul central al universității, site-ul de admitere și site-urile facultăților. Fiecare site are propriile meniuri, propriile convenții de denumire și propriul ritm de actualizare. O întrebare aparent simplă, de exemplu „Unde găsesc orarul?” sau „Se pot cumula bursele?”, poate necesita parcurgerea mai multor pagini înainte de obținerea unei surse oficiale potrivite. Problema nu este lipsa informației, ci costul de acces la informația corectă.

Lucrarea pornește de la această observație și propune un asistent integrat în browser, capabil să răspundă la întrebări formulate natural și să indice sursele oficiale folosite. Integrarea sub forma unei extensii Chrome este importantă deoarece păstrează asistentul în același context în care apare nevoia de informare. Studentul nu este obligat să deschidă o aplicație separată, ci poate interoga sistemul direct din browser.

1.2 Motivație

Asistenții conversaționali bazați pe modele de limbaj au devenit o soluție atractivă pentru interacțiunea în limbaj natural. Totuși, într-un context instituțional, un răspuns fluent nu este suficient. Studentul are nevoie de informații verificabile, iar sistemul

trebuie să evite răspunsurile speculative. În domenii precum bursele, admiterea, regulamentele sau procedurile administrative, o formulare aproximativă poate produce confuzie.

Din acest motiv, UVT Asist nu tratează modelul de limbaj ca sursă unică de adevăr. Modelul este folosit ca instrument de formulare, iar informația factuală este extrasă din pagini oficiale. Abordarea este apropiată de paradigma Retrieval-Augmented Generation, în care generarea răspunsului este condiționată de context recuperat dintr-o colecție de documente [LPP⁺20]. Această decizie permite păstrarea avantajelor interfeței conversaționale, dar reduce dependența de cunoștințele interne ale modelului.

1.3 Scopul și obiectivele lucrării

Scopul lucrării este proiectarea și realizarea unui prototip funcțional pentru un asistent digital care facilitează accesul la informațiile oficiale UVT. Prototipul urmărește să fie util în situații frecvente pentru studenți: găsirea orarului, identificarea datelor de contact, orientarea către informații despre admitere, consultarea paginilor despre burse, regulamente sau alte resurse administrative.

Obiectivele principale sunt:

- proiectarea unei arhitecturi locale, în care extensia Chrome reprezintă interfața principală, iar backend-ul Flask orchestrează căutarea și generarea răspunsului;
- construirea unui index local din pagini oficiale UVT și din paginile facultăților;
- utilizarea embeddings-urilor și a unei baze vectoriale pentru regăsire semantică;
- filtrarea rezultatelor în funcție de facultate și de tipul paginii;
- combinarea scorurilor semantice cu reguli deterministe de reranking pentru a prioritiza sursele potrivite;
- generarea unor răspunsuri concise, în limba română, însoțite de sursele oficiale folosite;
- tratarea explicită a cazurilor cu încredere scăzută, fără a inventa informații nesustținute de surse.

1.4 Contribuții

Contribuția principală a lucrării constă în proiectarea unei soluții aplicate, adaptate ecosistemului UVT, care combină o interfață ușor de folosit cu un mecanism local

de retrieval și generare. Față de o simplă căutare pe site, sistemul acceptă întrebări naturale, corectează unele formulări imperfecte și prioritizează sursele relevante. Față de un chatbot generic, sistemul păstrează legătura cu sursele oficiale și comunică nivelul de încredere al răspunsului.

La nivel tehnic, proiectul propune un pipeline complet: descoperirea paginilor, extragerea textului, clasificarea paginilor, segmentarea în fragmente, construirea reprezentărilor vectoriale, stocarea în Qdrant, căutarea semantică, reranking-ul determinist, verificarea live limitată și generarea locală cu Ollama. La nivel de interacțiune, extensia oferă selecția facultății, istoricul conversației, întrebări recente, afișarea surselor și stări clare pentru backend indisponibil sau indexare în desfășurare.

1.5 Structura lucrării

Capitolul al doilea prezintă fundamentele teoretice relevante: regăsirea informației, modelele de limbaj, embeddings-urile, căutarea vectorială și paradigma Retrieval-Augmented Generation. Capitolul al treilea analizează problema concretă din ecosistemul UVT și formulează cerințele sistemului. Capitolul al patrulea descrie arhitectura aplicației, iar capitolul al cincilea detaliază componentele implementate. Capitolul al șaselea discută validarea, limitele și riscurile soluției. Ultimele capitole prezintă concluziile și activitatea desfășurată.



Figura 1.1: Exemplu de context web în care utilizatorul poate avea nevoie de asistență pentru navigare

Capitolul 2

Fundamente teoretice

2.1 Regăsirea informației

Regăsirea informației este domeniul care studiază identificarea documentelor relevante pentru o nevoie de informare exprimată printr-o interogare. În sistemele clasice, documentele sunt indexate prin termeni, iar potrivirea se face prin măsuri lexicale. Modelele de tip TF-IDF sau BM25 valorifică frecvența termenilor și raritatea lor în colecție pentru a estima relevanța [MRS08, RZ09].

Aceste metode au avantaje importante: sunt eficiente, interpretabile și nu necesită infrastructură complexă. Totuși, ele depind mult de suprapunerea exactă dintre termenii întrebării și termenii documentului. Într-un sistem destinat studenților, această dependență poate fi problematică. Utilizatorii pot întreba „unde găsesc programul?” în loc de „orar”, pot scrie cu greșeli sau pot folosi forme flexionare diferite. De aceea, un sistem robust trebuie să includă normalizare, corecție de termeni și semnale suplimentare din titlu, URL și tipul paginii.

În UVT Asist, căutarea lexicală nu este eliminată. Ea rămâne importantă pentru reranking și fallback, deoarece oferă semnale precise pentru întrebări de navigare. De exemplu, apariția cuvântului „orar” în URL-ul unei pagini oficiale este un indiciu foarte puternic. În schimb, pentru întrebări mai libere, precum cele despre cumularea burselor, căutarea semantică ajută la identificarea documentelor relevante chiar dacă formularea exactă diferă.

2.2 Modele de limbaj și limitele lor

Modelele de limbaj moderne sunt construite pentru a prezice și genera text. Arhitectura Transformer a devenit baza multor modele actuale deoarece mecanismul de atenție permite modelarea eficientă a relațiilor dintre cuvinte pe distanțe mari [VSP⁺17]. Aceste modele pot produce răspunsuri fluente și pot înțelege într-o anumită măsură

intenția utilizatorului, ceea ce le face utile pentru interfețe conversaționale.

În același timp, un model de limbaj nu garantează corectitudinea factuală a răspunsului. Dacă este întrebat despre o regulă administrativă și nu primește context oficial, modelul poate genera un răspuns plauzibil, dar greșit sau depășit. Această problemă este cunoscută în practică drept halucinație. Într-o aplicație universitară, riscul este semnificativ deoarece utilizatorul poate lua decizii pe baza răspunsului.

Pentru UVT Asist, concluzia teoretică este clară: modelul generativ trebuie constrâns de surse. Rolul lui nu este să decidă ce pagină este oficială și nici să inventeze reguli, ci să formuleze un răspuns scurt pe baza fragmentelor selectate de backend. De aceea, selecția surselor este realizată determinist de componentele de retrieval și reranking, nu de modelul de limbaj.

2.3 Embeddings și căutare vectorială

Embeddings-urile sunt reprezentări numerice dense ale textului. Ideea de bază este că texte cu sens apropiat sunt mapate în zone apropiate ale unui spațiu vectorial. O întrebare și un fragment de document pot fi comparate prin similaritate cosinus:

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|},$$

unde q este vectorul întrebării, iar d este vectorul fragmentului de document.

Căutarea vectorială este utilă deoarece poate recupera documente relevante chiar și atunci când termenii nu coincid exact. Într-o colecție universitară, aceeași nevoie poate fi exprimată diferit: „taxe”, „achitare”, „plată”, „secretariat”, „program cu publicul” sau „contact”. Embeddings-urile pot aproxima aceste relații semantice mai bine decât o simplă potrivire textuală.

Pentru stocarea și căutarea embeddings-urilor, proiectul folosește Qdrant, o bază de date vectorială care permite căutare după similaritate și filtrare pe metadata [Qdr]. Filtrarea este esențială în acest proiect: aceeași întrebare poate avea răspunsuri diferite pentru Facultatea de Informatică și pentru nivelul general UVT. Prin urmare, fiecare fragment indexat conține metadata precum `faculty_id`, `page_type`, `title`, `url` și `last_indexed`.

2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation combină două etape: recuperarea documentelor relevante și generarea răspunsului pe baza acestora [LPP⁺20]. În loc ca modelul să răspundă doar din parametrii săi interni, sistemul îi oferă un context explicit. Această

abordare este potrivită pentru domenii în care informația se schimbă sau trebuie justificată prin surse.

Un flux RAG tipic conține următorii pași:

1. documentele sunt colectate și segmentate în fragmente;
2. fragmentele sunt transformate în embeddings și indexate;
3. întrebarea utilizatorului este normalizată și transformată la rândul ei într-un embedding;
4. sistemul recuperează fragmentele relevante;
5. fragmentele sunt ordonate și filtrate;
6. modelul de limbaj primește întrebarea și contextul oficial;
7. răspunsul este returnat împreună cu sursele folosite.

Această schemă nu elimină complet riscurile. Dacă etapa de retrieval selectează surse slabe, modelul poate formula un răspuns slab. Dacă documentele indexate sunt depășite, răspunsul poate fi depășit. De aceea, UVT Asist adaugă mecanisme suplimentare: clasificarea tipului de întrebare, reranking determinist, scor de încredere, fallback lexical, răspunsuri deterministe pentru întrebări de navigare și verificare live limitată a surselor de top.

2.5 Extensii de browser ca interfață

O extensie de browser este potrivită pentru aplicații care trebuie să fie disponibile rapid în timpul navigării. Chrome Manifest V3 definește structura extensiilor moderne, inclusiv fișierul manifest, popup-ul, permisiunile și accesul la API-uri de browser [Goo]. În cazul UVT Asist, extensia nu modifică paginile UVT și nu injectează logică în site-uri. Ea oferă un popup separat care comunică local cu backend-ul Flask.

Această alegere are două avantaje. Primul este ergonomia: utilizatorul poate pune întrebări fără a părăsi browserul. Al doilea este separarea responsabilităților: extensia rămâne responsabilă pentru interfață, iar backend-ul pentru retrieval, generare, cache și feedback. Comunicarea se face cu un server local, disponibil pe 127.0.0.1:5000.

2.6 Rulare locală și confidențialitate

O decizie importantă a proiectului este folosirea componentelor AI locale. Ollama este utilizat atât pentru generarea răspunsurilor, cât și pentru embeddings [Oll]. Această

abordare reduce dependența de servicii externe și permite demonstrarea aplicației într-un mediu controlat. În plus, întrebările utilizatorului și fragmentele recuperate nu trebuie trimise către un API remote.

Rularea locală implică însă costuri. Performanța depinde de hardware, modelele trebuie instalate pe mașina locală, iar Qdrant și Flask trebuie pornite înainte de utilizarea extensiei. De aceea, aplicația include endpoint-uri de stare și mesaje de interfață pentru backend indisponibil, indexare în desfășurare, Ollama indisponibil sau Qdrant indisponibil.

2.7 Calitatea surselor și prospețimea informației

Într-un sistem conversațional obișnuit, calitatea răspunsului este evaluată în primul rând prin coerență și utilitate. Într-un sistem instituțional, aceste criterii nu sunt suficiente. Răspunsul trebuie să fie susținut de o sursă oficială, iar sursa trebuie să fie suficient de recentă pentru întrebarea adresată. De exemplu, o pagină veche despre admiterea dintr-un an anterior poate conține termeni foarte apropiați de întrebarea utilizatorului, dar nu trebuie preferată în fața unei pagini stabile despre procesul curent de admitere.

Această problemă este una de selecție a dovezilor. Un model de limbaj poate formula un răspuns fluent dintr-un fragment nepotrivit, ceea ce face ca etapa de retrieval să fie critică. În UVT Asist, paginile oficiale sunt tratate diferit în funcție de tip. Paginile de navigare, precum cele de orar sau contact, sunt de regulă mai stabile. Documentele de metodologie și regulament au autoritate mai mare pentru întrebări normative. Știrile și anunțurile datate pot fi utile, dar trebuie penalizate atunci când întrebarea cere o resursă generală.

Prospețimea informației este abordată prin două mecanisme complementare. Primul este reconstrucția indexului, care actualizează snapshot-ul local al paginilor oficiale. Al doilea este verificarea live limitată, care poate reextrage conținutul pentru sursele de top înainte de formularea răspunsului. Verificarea live nu înlocuiește indexul local, deoarece o căutare live largă ar fi mai lentă și mai greu de controlat. Ea funcționează ca un pas de confirmare pentru un număr mic de pagini deja selectate.

2.8 Degradare controlată

O aplicație bazată pe mai multe componente locale trebuie să accepte faptul că unele servicii pot lipsi temporar. Ollama poate să nu fie pornit, modelul de embedding poate să nu fie instalat, Qdrant poate să nu aibă colecția construită, iar indexarea poate fi în desfășurare. Într-o astfel de situație, un sistem robust nu trebuie să eșueze

necontrolat și nici să afișeze utilizatorului un răspuns aparent normal construit din date insuficiente.

UVT Asist aplică principiul degradării controlate. Dacă Qdrant sau embeddings-urile nu sunt disponibile, backend-ul poate folosi fallback lexical pe indexul JSON. Dacă dovezile sunt prea slabe, generarea cu modelul de limbaj este evitată. Dacă indexarea de startup rulează, endpoint-ul de chat returnează un mesaj explicit cu progresul indexării. Dacă backend-ul nu răspunde, extensia afișează starea de indisponibilitate în loc să lase utilizatorul fără explicație.

Această abordare este importantă din două motive. În primul rând, crește predictibilitatea aplicației în timpul demonstrației și dezvoltării. În al doilea rând, respectă natura domeniului: pentru informații administrative, este mai acceptabil să spui „nu am suficiente dovezi oficiale” decât să produci un răspuns nesuținut.

Capitolul 3

Analiza problemei și cerințe

3.1 Particularitățile ecosistemului UVT

Ecosistemul web al UVT este format din mai multe surse oficiale: site-ul central, site-ul de admitere și site-urile facultăților. Din perspectiva unui sistem automat, această structură produce trei dificultăți. Prima este dispersia informației: unele răspunsuri sunt la nivelul universității, altele la nivelul facultății. A doua este eterogenitatea: paginile pot fi HTML, PDF, DOCX, text sau imagini care necesită OCR. A treia este dinamica informației: anumite pagini se schimbă periodic, în special cele legate de admitere, orare, burse sau calendare.

Un student nu formulează întrebări în termenii interni ai site-ului. El poate întreba „care e programul secretariatului?”, „unde văd orarul la info?”, „pot primi două burse?” sau „unde mă înscriu la admitere?”. Sistemul trebuie să transforme aceste formulări într-o căutare controlată, să aleagă domeniul potrivit și să returneze sursa oficială relevantă.

3.2 Cerințe funcționale

Cerințele funcționale urmărite în proiect sunt:

- utilizatorul poate selecta facultatea relevantă sau poate lucra în contextul general UVT;
- utilizatorul poate trimite întrebări în limbaj natural din popup-ul extensiei;
- backend-ul detectează intenții pentru orar, contact, burse, admitere, regulamente, informații pentru studenți sau întrebări generale;
- sistemul recuperează fragmente relevante din indexul local și le ordonează după scor;

- răspunsul include surse oficiale distincte, afișate clar în interfață;
- pentru întrebări ambigue, sistemul poate cere alegerea unei facultăți;
- aplicația expune starea componentelor prin endpoint-uri de health și status;
- utilizatorul poate oferi feedback asupra răspunsului primit.

3.3 Cerințe nefuncționale

Pe lângă funcționalități, proiectul impune cerințe nefuncționale importante. Sistemul trebuie să favorizeze sursele oficiale, să evite afirmațiile nesustținute, să funcționeze local, să degradeze controlat atunci când Qdrant sau Ollama nu sunt disponibile și să rămână utilizabil chiar dacă indexarea este în curs.

O altă cerință este transparența. Utilizatorul trebuie să vadă sursele, iar backend-ul trebuie să atașeze metadate despre încredere, modul de retrieval, verificarea live și modul de generare. Această transparență este esențială într-un sistem educațional, deoarece răspunsurile nu trebuie tratate ca autoritate finală fără posibilitatea de verificare.

3.4 Categoriile de întrebări

Analiza proiectului arată că întrebările studenților pot fi împărțite în mai multe categorii:

- întrebări de navigare, unde răspunsul este de obicei un link oficial, de exemplu pagina de orare sau pagina de contact;
- întrebări administrative, unde este necesară sinteza scurtă a unei pagini sau metodologii;
- întrebări de regulament, unde sursa trebuie să fie o metodologie sau un document instituțional;
- întrebări dependente de facultate, unde aceeași formulare poate conduce la pagini diferite;
- întrebări vagi sau incomplete, unde sistemul trebuie să evite alegerea arbitrară a unei surse.

Această clasificare influențează arhitectura. Pentru întrebările de navigare, răspunsurile deterministe sunt mai sigure și mai rapide decât generarea cu LLM. Pentru

întrebările administrative, modelul poate fi util dacă există dovezi suficiente. Pentru întrebările de regulament, reranking-ul trebuie să favorizeze documentele de tip metodologie, procedură sau regulament.

3.5 Cazuri de utilizare reprezentative

Un caz de utilizare frecvent este căutarea orarului. Studentul nu are nevoie de o explicație lungă despre organizarea cursurilor, ci de pagina oficială pe care poate consulta orarul. În acest caz, sistemul trebuie să identifice facultatea, să favorizeze URL-ul specific și să returneze un răspuns scurt. Pentru Facultatea de Informatică, pagina de orare are un semnal puternic atât în URL, cât și în titlu, ceea ce justifică un răspuns determinist.

Un al doilea caz de utilizare este identificarea secretariatului sau a datelor de contact. Întrebările din această categorie pot include termeni precum „program”, „telefon”, „email”, „secretariat” sau „contact”. Răspunsul poate depinde de facultatea selectată, iar alegerea unei pagini generale UVT poate fi insuficientă. De aceea, sistemul tratează contactul ca intenție dependentă de facultate și poate cere clarificare dacă utilizatorul lucrează în contextul general.

Un caz mai dificil este întrebarea de tip regulament, de exemplu „Se pot cumula bursele?”. Aici nu este suficientă o pagină informativă generică despre burse. Sistemul trebuie să favorizeze metodologii, regulamente sau documente instituționale, deoarece răspunsul depinde de formulări oficiale. În plus, răspunsul trebuie să fie prudent: dacă fragmentul recuperat nu conține regula relevantă, aplicația trebuie să indice surse apropiate, nu să deducă o regulă.

Un alt caz important este admiterea. Site-urile de admitere conțin adesea știri anuale, comunicate și pagini stabile despre proces. Pentru întrebarea „Unde găsesc informații despre admitere?”, o pagină stabilă despre procesul de preînscrisere este mai potrivită decât o știre veche. Acest caz a influențat direct reranking-ul, prin penalizarea știrilor datate atunci când întrebarea nu menționează explicit un an.

3.6 Riscuri informaționale

Riscul principal într-un asistent instituțional este producerea unei concluzii nejustificate. Un răspuns incorrect despre burse, admitere sau proceduri poate avea consecințe mai mari decât o simplă recomandare greșită. Din acest motiv, sistemul trebuie să trateze informația administrativă ca informație cu sensibilitate ridicată. În proiect, această decizie se reflectă în reguli de prompt, în scorul de încredere și în alegerea de a nu apela modelul generativ atunci când nu există dovezi.

Un al doilea risc este alegerea unei surse oficiale, dar nepotrivite. O pagină a unei facultăți poate fi oficială, însă nu răspunde întrebării generale. O metodologie veche poate fi oficială, dar nu neapărat actuală. O știre despre admitere poate fi oficială, dar depășită. De aceea, proiectul nu folosește doar criteriul „domeniu oficial”, ci combină domeniul cu tipul paginii, titlul, URL-ul, conținutul și intenția detectată.

Un al treilea risc este formularea ambiguă a întrebărilor. Dacă utilizatorul întreabă doar „program” sau „detalii”, sistemul nu poate ști dacă se referă la orar, program cu publicul sau un program de studii. În astfel de situații, clarificarea este preferabilă unei alegeri arbitrare. Această idee este importantă pentru experiența utilizatorului: un asistent bun nu trebuie să pară sigur atunci când întrebarea nu oferă suficient context.

3.7 Criterii de acceptare

Pentru ca prototipul să fie considerat funcțional, au fost stabilite câteva criterii de acceptare. Sistemul trebuie să pornească local, să expună starea componentelor, să încarce lista de facultăți și să răspundă la întrebări reprezentative. Pentru întrebările de orar și contact, sursa de top trebuie să fie pagina specifică facultății. Pentru întrebările de burse și regulamente, rezultatele trebuie să favorizeze documente sau pagini instituționale cu autoritate.

În plus, aplicația trebuie să ofere răspunsuri transparente. Fiecare răspuns trebuie să includă surse oficiale atunci când există dovezi, iar nivelul de încredere trebuie să fie vizibil. În cazul lipsei de dovezi, sistemul trebuie să returneze un mesaj de limitare. Pentru demonstrație, trebuie să fie prezentabile și stările de eșec: backend indisponibil, indexare în curs, Ollama indisponibil, Qdrant indisponibil și retrieval slab.

Capitolul 4

Arhitectura sistemului UVT Asist

4.1 Viziune generală

UVT Asist este construit ca sistem local, compus dintr-o extensie Chrome, un backend Flask, un index JSON, o bază vectorială Qdrant și modele Ollama pentru embeddings și generare. Produsul vizibil pentru utilizator este extensia, iar restul componentelor rulează local și susțin procesarea întrebării.

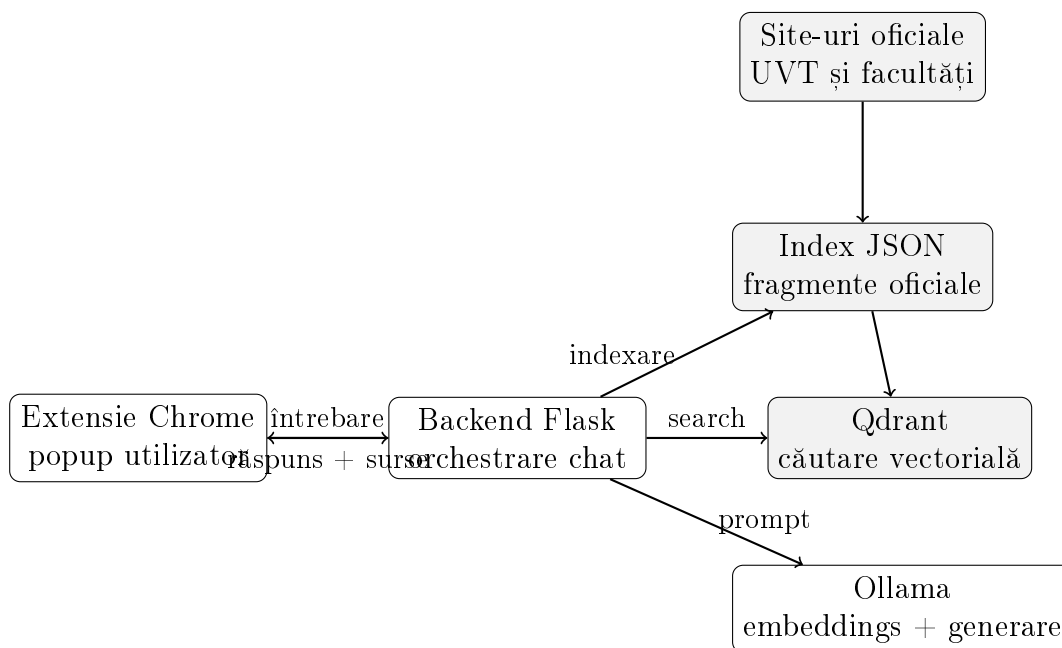


Figura 4.1: Arhitectura logică a sistemului UVT Asist

Separarea componentelor reduce complexitatea interfeței și permite dezvoltarea independentă a fiecărui nivel. Extensia nu trebuie să cunoască detaliile indexului sau ale modelului de limbaj. Backend-ul nu depinde de browser pentru logica de retrieval. Qdrant poate fi înlocuit sau reconstruit fără modificarea popup-ului, iar modelul Ollama poate fi schimbat prin configurare.

4.2 Principii de proiectare

Arhitectura a fost ghidată de câteva principii. Primul principiu este separarea clară între interfață și procesarea informației. Extensia Chrome trebuie să rămână simplă, predictibilă și ușor de folosit. Ea nu trebuie să implementeze logica de căutare, normalizare, scorare sau generare. Aceste responsabilități aparțin backend-ului, unde pot fi testate și modificate mai ușor.

Al doilea principiu este controlul asupra surselor. Sistemul nu folosește surse externe neoficiale pentru generarea răspunsurilor. Domeniile permise sunt definite în configurația facultăților, iar crawler-ul filtrează link-urile pentru a rămâne în aceste domenii. Această alegere reduce acoperirea posibilă, dar crește încrederea în sursele afișate.

Al treilea principiu este combinarea metodelor probabilistice cu reguli deterministe. Embeddings-urile și căutarea vectorială sunt utile pentru flexibilitate semantică, dar nu sunt suficiente pentru un sistem instituțional. De aceea, rezultatele sunt rerankate folosind semnale explicite: potrivirea facultății, tipul paginii, URL-ul, titlul, termenii de regulament și penalizările pentru pagini generice.

Al patrulea principiu este localitatea. În stadiul de prototip, rularea locală permite control asupra datelor și reduce dependența de servicii remote. Această decizie este potrivită pentru o lucrare de licență deoarece face sistemul demonstrabil pe o configurație reproductibilă: Flask, Ollama, Qdrant și extensia Chrome.

4.3 Modelul datelor

Unitatea principală a sistemului nu este pagina întreagă, ci fragmentul de pagină. Paginile web pot conține mult text irelevant, iar unele documente oficiale sunt lungi. Segmentarea în fragmente permite o căutare mai fină și reduce cantitatea de context trimisă modelului generativ. Fiecare fragment este salvat atât în indexul JSON, cât și ca payload în Qdrant.

Câmp	Rol în sistem
chunk_id	Identificator stabil pentru fragment, folosit la deduplicare și la stocarea punctelor vectoriale.
faculty_id	Leagă fragmentul de contextul general UVT sau de o facultate concretă.
page_type	Clasifică pagina ca orar, contact, admitere, burse, regulamente, studenți sau general.
title	Oferă un semnal lexical puternic și este afișat în sursele returnate utilizatorului.
url	Păstrează legătura cu pagina oficială și permite deduplicarea surselor.
chunk_text	Conține textul efectiv folosit pentru retrieval, verificare și generare.
last_indexed	Indică momentul la care fragmentul a fost introdus în index.

Tabela 4.1: Câmpurile principale ale unui fragment indexat

Acest model de date are avantajul că păstrează o legătură directă între răspuns și sursă. Chiar dacă un fragment este recuperat prin similaritate vectorială, backend-ul poate afișa URL-ul exact al paginii și poate calcula semnale suplimentare pe titlu și conținut. În plus, indexul JSON rămâne inspectabil manual, ceea ce este util în dezvoltare și debugging.

4.4 Fluxul de indexare

Fluxul de indexare construiește baza de cunoștințe locală. Pentru fiecare sursă oficială configurată, sistemul pornește de la URL-uri de bază și de la rute prioritare, precum `/orare/`, `/burse/`, `/contact/`, `/admitere/`, `/regulamente/` sau `/metodologie/`. Dacă sunt disponibile sitemap-uri, acestea sunt folosite pentru descoperire mai largă. Apoi, link-urile candidate sunt filtrate astfel încât să rămână în domeniile oficiale permise.

Paginile acceptate sunt descărcate și convertite în text. Pentru HTML se elimină elementele irelevante precum meniuri, scripturi, formulare sau bannere. Pentru PDF și DOCX se extrage textul cu biblioteci dedicate, iar pentru imagini sau PDF-uri scanate există suport OCR opțional. Textul este limitat și împărțit în fragmente, pentru ca fiecare fragment să poată fi comparat eficient cu întrebările utilizatorilor.

Fiecare fragment primește metadate: identificator, facultate, tip de pagină, titlu, URL, text și momentul indexării. Documentul rezultat este salvat ca index JSON, iar fragmentele sunt transformate în embeddings și încărcate în Qdrant. Astfel, sistemul păstrează atât o reprezentare textuală inspectabilă, cât și una vectorială optimizată pentru căutare semantică.

La nivel operațional, indexarea poate rula manual sau la pornirea backend-ului. Pentru demonstrații, reconstrucția completă a indexului poate fi pornită înaintea prezentării, iar extensia afișează progresul. Această funcționalitate este utilă deoarece un index incomplet poate induce răspunsuri greșite. În timpul indexării, endpoint-ul de chat este blocat controlat și returnează un mesaj temporar, în loc să răspundă dintr-o stare parțială.

4.5 Fluxul de interogare

La primirea unei întrebări, backend-ul normalizează textul, aplică reguli pentru corectarea unor greșeli frecvente și detectează intenția. Dacă utilizatorul a selectat o facultate, aceasta devine filtru principal. Dacă a selectat contextul general UVT, backend-ul poate încerca să deducă facultatea din întrebare sau din istoricul recent.

Întrebarea este transformată în embedding cu Ollama și trimisă către Qdrant. Căutarea vectorială folosește filtre pe `faculty_id` și `page_type`. Pentru robustețe, sistemul execută mai multe treceri de căutare: întâi cu filtre stricte, apoi cu filtre mai largi. Rezultatele semantice sunt completate printr-un backfill lexical, util când vectorii nu surprind suficient semnalele explicite din URL sau titlu.

După recuperare, candidații sunt reordonați determinist. Scorul final combină suprapuneri lexicale, potrivirea facultății, preferința pentru tipul paginii, semnale din URL, penalizări pentru pagini generice și bonusuri pentru documente de regulament sau metodologie. Rezultatul final este un set mic de surse diverse, de regulă cel mult un fragment per URL.

Algorithm 1 Procesarea unei întrebări în UVT Asist

Require: întrebare, facultate selectată, istoric conversațional

Ensure: răspuns, surse oficiale, scor de încredere

```
1: normalizează întrebarea și corectează termeni frecvenți
2: determină facultatea efectivă și intenția întrebării
3: if întrebarea necesită clarificarea facultății then
4:     returnează mesaj de clarificare
5: end if
6: caută candidați în Qdrant cu filtre pe facultate și tip de pagină
7: completează rezultatele cu fallback lexical din indexul JSON
8: calculează scorurile deterministe și selectează surse diverse
9: verifică live sursele de top, dacă funcția este activată
10: calculează nivelul de încredere
11: if întrebarea este de navigare și încrederea este suficientă then
12:     construiește răspuns determinist
13: else if dovezile sunt slabe then
14:     returnează mesaj de limitare
15: else
16:     construiește promptul RAG și apelează modelul Ollama
17: end if
18: returnează răspunsul, sursele și metadatele
```

4.6 Controlul generării

Generarea cu modelul de limbaj este folosită doar când există dovezi suficiente. Pentru întrebări de navigare, precum orar, contact sau admitere, sistemul poate construi răspunsul direct, fără LLM. Pentru întrebări cu încredere scăzută, backend-ul returnează un mesaj de limitare și sursele apropiate, evitând generarea unui text care ar putea părea mai sigur decât este.

Când modelul este folosit, promptul include facultatea selectată, nivelul de încredere, intenția detectată, istoricul recent și fragmentele oficiale recuperate. Promptul impune răspuns în limba română, interzice explicațiile interne și cere folosirea exclusivă a contextului oficial. Dacă modelul produce un răspuns cu semne de comportament nedorit, backend-ul revine la un răspuns local determinist.

4.7 Verificare live și cache

Indexul local oferă viteză și stabilitate, dar paginile oficiale se pot schimba. Pentru a reduce acest risc, sistemul poate reverifica live un număr mic de surse de top. Verificarea live este intenționat limitată pentru a menține timpul de răspuns previzibil și pentru a evita comportamentul nedeterminist al unei căutări web largi. Rezultatele sunt păstrate temporar în cache, iar interfața marchează dacă sursele principale au

fost verificate live.

Capitolul 5

Componente și funcționalități

5.1 Structura proiectului

Proiectul este organizat în două zone principale: **backend/** și **extension/**. Directorul **backend/** conține serverul Flask, logica de indexare, retriever-ul, integrarea cu Ollama, integrarea cu Qdrant, extragerea textului din pagini și testele automate. Directorul **extension/** conține fișierul **manifest.json**, interfața popup-ului și logica JavaScript care comunică cu backend-ul local.

Această structură este potrivită pentru o aplicație de tip extensie browser cu servicii locale. Frontend-ul rămâne ușor de instalat în Chrome, iar backend-ul poate fi rulat și testat independent. În plus, există și o interfață HTML de test, folosită doar pentru dezvoltare, care permite apelarea endpoint-urilor fără încărcarea extensiei în browser.

Componentă	Responsabilitate
<code>app.py</code>	Expune API-ul Flask, orchestrează chat-ul, verifică starea componentelor și salvează feedback.
<code>build_index.py</code>	Descoperă pagini oficiale, descarcă surse și construiește indexul local.
<code>retriever.py</code>	Normalizează întrebarea, detectează intenții, caută în Qdrant și rerankează candidații.
<code>page_index.py</code>	Definește schema indexului, clasifică pagini și segmentează textul în fragmente.
<code>vector_store.py</code>	Inițializează Qdrant, creează indexuri de payload și execută căutări vectoriale.
<code>popup.js</code>	Gestionează conversația, stările UI, istoricul, feedback-ul și apelurile către backend.

Tabela 5.1: Structura principală a proiectului UVT Asist

5.2 Extensia Chrome

Extensia Chrome este interfața principală a aplicației. Ea oferă un popup cu selector de facultate, câmp pentru întrebare, afișare de mesaje, surse oficiale, nivel de încredere, indicator de verificare live și stare a backend-ului. Extensia păstrează istoricul conversației pe facultate și o listă scurtă de întrebări recente folosind mecanismul de stocare locală al browserului.

Din punct de vedere al produsului, această interfață este potrivită pentru scenariul de utilizare. Studentul poate întreba rapid fără a naviga manual prin mai multe meniuri. Din punct de vedere tehnic, extensia trimite cereri HTTP către backend-ul local, citește endpoint-ul `/health`, încarcă lista de facultăți din `/faculties` și trimite întrebările la `/chat`.

Popup-ul nu încearcă să ascundă starea tehnică a sistemului. Dacă backend-ul este disponibil și indexul este pregătit, utilizatorul vede o stare de sistem pregătit. Dacă indexarea rulează, interfața afișează progresul. Dacă backend-ul nu răspunde, butoanele rămân controlate, iar mesajul indică faptul că serviciul local trebuie pornit. Această vizibilitate este importantă deoarece prototipul depinde de mai multe procese locale.

5.3 Backend-ul Flask

Backend-ul Flask este componenta de orchestrare. Flask este un framework web Python minimalist, potrivit pentru prototipuri și servicii API locale [Pal]. În proiect, backend-ul expune endpoint-uri pentru stare, listă de facultăți, status de indexare, chat și feedback.

Endpoint-ul `/health` raportează disponibilitatea Ollama, modelele configurate, starea indexului JSON, starea colecției Qdrant, modul de retrieval, cache-urile și progresul indexării. Endpoint-ul `/indexing/status` permite extensiei să afișeze progresul indexării. Endpoint-ul `/feedback` salvează feedback-ul utilizatorului într-un fișier JSONL, util pentru analiză și îmbunătățiri ulterioare.

Endpoint	Rol
/health	Returnează starea Ollama, Qdrant, index JSON, cache-uri, mod de retrieval și readiness.
/faculties	Returnează lista facultăților configurate pentru selectorul din extensie.
/indexing/status	Returnează progresul indexării de startup.
/chat	Primește întrebarea, facultatea și istoricul, apoi returnează răspunsul și sursele.
/feedback	Salvează votul utilizatorului și metadatele răspunsului pentru analiză ulterioară.

Tabela 5.2: Endpoint-urile principale ale backend-ului

Backend-ul aplică și validări defensive. Întrebările goale primesc răspuns dedicat, payload-urile invalide nu produc eroare internă, istoricul este limitat ca număr de mesaje și lungime, iar întrebarea este trunchiată la o dimensiune configurabilă. Aceste limite previn comportamente necontrolate și fac sistemul mai potrivit pentru demonstrație.

5.4 Indexarea și extragerea conținutului

Indexarea începe cu lista de surse oficiale și cu rute considerate prioritare. Pentru fiecare facultate sunt generate URL-uri candidate din baza configurată, din sitemap-uri și din link-uri descoperite în paginile HTML. Crawler-ul acceptă pagini HTML, documente PDF, DOCX, fișiere text și, opțional, imagini pentru OCR. Link-urile sunt filtrate astfel încât să rămână pe domeniile oficiale configurate.

Extragerea textului diferă în funcție de tipul resursei. Pentru HTML, sistemul elimină meniuri, navigații, formulare, bannere, scripturi și alte elemente care pot introduce zgomot. Pentru PDF-uri se folosește extragerea de text, cu fallback OCR opțional atunci când documentul pare scanat. Pentru DOCX se extrag paragrafe și tabele. Rezultatul este normalizat și limitat ca dimensiune pentru a evita consumul excesiv de memorie.

Clasificarea paginilor se face prin indicii din URL, titlu și conținut. De exemplu, o pagină cu **/orare/** în URL și cu termenul „orar” în titlu este clasificată ca pagină de orar. O pagină care conține termeni precum „metodologie”, „regulament” sau „procedură” primește prioritate pentru tipul **regulamente**. Clasificarea nu trebuie să fie perfectă, dar trebuie să ofere semnale suficient de bune pentru filtrare și reranking.

5.5 Configurația facultăților

Aplicația conține o configurație a surselor oficiale. Aceasta include contextul general UVT, site-ul de admitere și site-urile facultăților. Fiecare intrare are un identificator scurt, un nume afișabil și o listă de URL-uri de bază. Această structură permite filtrarea rezultatelor și prezentarea unei liste clare în extensie.

Pentru Facultatea de Informatică, de exemplu, întrebările despre orar trebuie să favorizeze pagina `info.uvt.ro/orare`. Pentru întrebările generale despre burse, sistemul trebuie să favorizeze documente sau pagini UVT, nu neapărat o pagină a unei facultăți. Această diferențiere justifică existența câmpului `faculty_id` în fiecare fragment indexat.

Configurația nu este doar o listă pentru interfață. Ea definește spațiul permis de căutare. Dacă o facultate are mai multe domenii oficiale, acestea pot fi adăugate ca URL-uri de bază. Dacă o întrebare este adresată în contextul unei facultăți, sistemul caută mai întâi în sursele acelei facultăți și apoi poate lua în calcul sursele UVT generale, mai ales pentru regulamente sau metodologii aplicabile tuturor studenților.

5.6 Retriever-ul

Retriever-ul este componenta care transformă întrebarea în surse candidate. El realizează normalizarea diacriticelor, eliminarea diferențelor de formă, corectarea unor greșeli frecvente și detectarea intenției. De exemplu, formulări precum „orarul” sau „fac de info” sunt mapate către concepte relevante pentru orar și informatică.

După analiza întrebării, retriever-ul construiește preferințe pentru tipurile de pagini. O întrebare despre „burse” poate prefera pagini de burse și regulamente, iar o întrebare despre cumulare de burse este tratată ca întrebare de metodologie sau regulament. Această clasificare reduce riscul ca o pagină generică să fie aleasă înaintea unei metodologii oficiale.

Scorarea finală este hibridă. Rezultatul semantic din Qdrant este combinat cu semnale deterministe: titlu, URL, text, facultate, tip de pagină, document oficial, întrebări de politici și penalizări pentru homepage-uri. Prin această combinație, sistemul păstrează flexibilitatea embeddings-urilor, dar rămâne controlabil.

5.7 Integrarea Ollama și Qdrant

Ollama este folosit în două momente diferite. În etapa de indexare, fiecare fragment este transformat într-un text de embedding care include titlul, URL-ul, facultatea, tipul paginii și conținutul oficial. Acest text este trimis modelului de embedding, iar

vectorul rezultat este încărcat în Qdrant. În etapa de chat, întrebarea utilizatorului este transformată într-un embedding pentru căutare semantică.

Qdrant stochează un punct vectorial pentru fiecare fragment. Pe lângă vector, fiecare punct conține payload-ul textual și metadatele necesare. Căutarea poate fi filtrată după `faculty_id` și `page_type`, ceea ce este esențial pentru reducerea spațiului de rezultate. Fără aceste filtre, o întrebare despre orar ar putea primi rezultate de la o altă facultate sau pagini generale care folosesc termeni similari.

Modelul generativ Ollama este apelat doar după selecția surselor. Promptul final conține contextul oficial și instrucțiuni stricte: răspuns în română, concizie, folosirea exclusivă a contextului, evitarea speculațiilor și atenție specială la întrebările de regulament. Astfel, modelul devine un strat de formulare, nu o autoritate factuală independentă.

5.8 Scorul de încredere

Fiecare set de rezultate primește un nivel de încredere: `high`, `medium` sau `low`. Scorul ține cont de puterea rezultatului principal, diferența față de următorul rezultat, numărul de surse distincte și semnalele de potrivire. Dacă lipsesc dovezi directe sau dacă sursa este prea generică, scorul este limitat.

Acest mecanism este important deoarece nu toate întrebările pot primi un răspuns sigur. În loc să ascundă incertitudinea, sistemul o comunică. Interfața afișează nivelul de încredere, iar backend-ul poate decide să nu invoce modelul de limbaj atunci când dovezile sunt slabe.

5.9 Răspunsuri, surse și feedback

Răspunsul trimis către extensie conține textul final, lista surselor, facultatea potrivită, scorul de încredere, profilul întrebării, modul de retrieval, modul de generare și profilul dovezilor. Sursele sunt deduplicate pe URL și afișate ca link-uri oficiale. Astfel, utilizatorul poate verifica imediat pagina pe care se bazează răspunsul.

Feedback-ul utilizatorului este salvat împreună cu întrebarea, răspunsul, sursele, scorul de încredere și modul de generare. Într-o etapă ulterioară, aceste date pot fi folosite pentru identificarea întrebărilor problematice, a surselor slab indexate sau a cazurilor în care reranking-ul trebuie ajustat.

Capitolul 6

Validare, limite și direcții viitoare

6.1 Strategia de validare

Validarea prototipului se face prin teste automate și prin scenarii manuale de demonstrație. Testele automate acoperă componente esențiale: corectarea întrebărilor, preferința pentru pagini stabile de admitere față de știri vechi, clasificarea paginilor, normalizarea URL-urilor, rezistența la payload-uri greșite și comportamentul sistemului când indexarea este în desfășurare.

Scenariile manuale verifică situații reprezentative pentru studenți:

- pentru Facultatea de Informatică, întrebarea „Unde găsesc orarul?” trebuie să favorizeze pagina oficială de orare;
- întrebarea despre secretariatul Facultății de Informatică trebuie să favorizeze pagina de contact;
- întrebările despre cumularea burselor trebuie să favorizeze metodologii sau regulamente oficiale;
- întrebările despre admitere trebuie să favorizeze paginile stabile ale procesului de admitere, nu știri vechi;
- formulările cu greșeli, precum „orarul”, trebuie să fie corectate suficient pentru a conduce la sursa potrivită;
- oprirea backend-ului trebuie să fie afișată clar în extensie.

6.2 Testarea automată

Testele automate sunt concentrate pe comportamente cu risc ridicat. O parte dintre teste verifică retriever-ul: corectarea termenilor greșiți, detectarea intențiilor și alegerea paginilor potrivite. De exemplu, testul pentru formularea „orarul” confirmă că

sistemul ajunge la intenția de orar, iar testul pentru admitere confirmă că o pagină stabilă despre procesul de admitere este preferată față de o știre veche.

O altă categorie de teste verifică indexul. Aceste teste validează normalizarea URL-urilor, detectarea tipului de pagină, existența câmpurilor necesare pentru payload-ul vectorial și limitarea dimensiunii fragmentelor. Limitarea fragmentelor este importantă deoarece paginile foarte mari sau textul fără spații pot produce probleme de memorie sau prompturi prea lungi.

Testele backend verifică robustețea API-ului. Un payload JSON greșit nu trebuie să producă o eroare 500. O întrebare cu dovezi slabe nu trebuie să invoce modelul de limbaj. O întrebare de navigare cu rezultat sigur trebuie să primească răspuns determinist. De asemenea, când indexarea este în desfășurare, chat-ul trebuie să se oprească temporar și să returneze statusul de indexare.

Această acoperire nu demonstrează că aplicația răspunde corect la orice întrebare, dar verifică proprietățile esențiale ale arhitecturii: sursele specifice sunt preferate, fallback-ul funcționează, generarea este controlată și stările de eșec sunt tratate explicit.

6.3 Evaluarea scenariilor manuale

Evaluarea manuală urmărește comportamentul perceput de utilizator. Pentru fiecare scenariu, se verifică trei aspecte: sursa de top, calitatea răspunsului și metadatele afișate. Sursa de top trebuie să fie oficială și potrivită. Răspunsul trebuie să fie scurt și să nu adauge informații nesusținute. Metadatele trebuie să indice facultatea potrivită, nivelul de încredere și modul de retrieval.

Pentru întrebările de orar, succesul înseamnă orientarea către pagina oficială de orare a facultății selectate. Pentru contact, succesul înseamnă pagina de contact sau secretariat. Pentru admitere, succesul înseamnă preferința pentru pagini stabile și nu pentru știri vechi. Pentru burse, succesul înseamnă găsirea unei metodologii sau a unei pagini instituționale relevante, nu doar a unei pagini generice despre studenți.

În scenariile cu dovezi slabe, succesul nu înseamnă neapărat un răspuns factual complet. Un rezultat corect poate fi un mesaj de tip „nu am găsit suficiente dovezi oficiale”. Această interpretare este importantă deoarece aplicația urmărește acuratețe și transparență, nu doar completarea conversației cu orice preț.

6.4 Metrici calitative

Pentru un prototip de licență, evaluarea poate fi formulată prin metrici calitative. Prima metrică este relevanța sursei de top: dacă primul link afișat este pagina pe care

un student ar trebui să o consulte. A doua metrică este acoperirea surselor: dacă lista de surse oferă cel puțin o alternativă oficială utilă. A treia metrică este fidelitatea răspunsului: dacă textul final poate fi susținut de fragmentele recuperate.

O altă metrică este comportamentul la incertitudine. Un sistem conversațional poate părea mai bun dacă răspunde mereu, dar pentru informații instituționale această proprietate este periculoasă. De aceea, un răspuns prudent, însoțit de surse apropiate, este preferabil unui răspuns detaliat fără dovezi. În UVT Asist, această metrică este susținută de scorul de încredere și de pragul sub care generarea este evitată.

Ultima metrică este stabilitatea demonstrației. Pentru o aplicație locală, utilizatorul trebuie să poată vedea rapid dacă sistemul este pregătit. Endpoint-ul de health, progresul indexării și mesajele de backend indisponibil contribuie la această stabilitate.

6.5 Limite

Sistemul depinde de calitatea indexului local. Dacă o pagină oficială lipsește din index sau dacă textul nu poate fi extras corect, răspunsul poate fi incomplet. Documentele PDF scanate pot necesita OCR, iar calitatea OCR-ului poate varia. De asemenea, dacă o pagină se modifică după indexare, sistemul poate folosi informații depășite până la reconstrucția indexului, cu excepția surselor verificate live.

O altă limită este performanța locală. Ollama și Qdrant trebuie să ruleze pe mașina utilizatorului sau pe o mașină de demonstrație. Timpul de răspuns depinde de modelul ales, de dimensiunea indexului și de hardware. Pentru o utilizare instituțională reală, ar putea fi necesară migrarea către un serviciu administrat intern, cu resurse controlate.

În plus, un scor de încredere este o aproximare, nu o garanție matematică a adevărului. El indică potrivirea dintre întrebare și sursele recuperate. Decizia finală pentru informații administrative importante trebuie să rămână verificabilă prin sursa oficială afișată.

6.6 Amenințări la validitate

O amenințare la validitate este dimensiunea setului de întrebări folosit pentru testare. Scenariile selectate acoperă întrebări importante, dar nu includ toate formulările posibile ale studenților. În practică, utilizatorii pot folosi abrevieri, greșeli diferite, întrebări compuse sau contexte conversaționale mai lungi. Pentru o evaluare mai solidă, ar fi necesar un set mai mare de întrebări colectate de la studenți.

O a doua amenințare este dependența de starea site-urilor oficiale în momentul indexării. Dacă o pagină este temporar indisponibilă sau dacă structura HTML se schimbă, crawler-ul poate extrage mai puțin conținut. De asemenea, o evaluare făcută

pe un index construit într-o anumită zi poate să nu reflecte perfect comportamentul după câteva luni.

O a treia amenințare este variația modelelor locale. Modelul de generare și modelul de embedding sunt configurabile. Schimbarea lor poate modifica atât calitatea răspunsurilor, cât și scorurile semantice. De aceea, atunci când modelul de embedding se schimbă, indexul vectorial trebuie reconstruit, iar scenariile de validare trebuie reluate.

6.7 Considerente de securitate și confidențialitate

Prototipul rulează local și nu trimite întrebările către un serviciu AI extern. Acest lucru este un avantaj pentru confidențialitate, dar nu elimină toate responsabilitățile. Backend-ul acceptă cereri HTTP locale, deci trebuie tratat ca un serviciu de dezvoltare, nu ca un server expus public. Într-o versiune instituțională, ar fi necesare autentificare, control de acces, rate limiting și monitorizare.

Feedback-ul este salvat local în format JSONL. Chiar dacă scopul lui este îmbunătățirea sistemului, el poate conține întrebări formulate liber de utilizatori. Într-o versiune distribuită, ar trebui stabilite reguli de retenție, anonimizare și informare a utilizatorilor. Pentru lucrarea de față, feedback-ul este tratat ca mecanism de dezvoltare și demonstrație.

Un alt aspect este integritatea surselor. Sistemul presupune că domeniile configurate sunt oficiale și că paginile accesate prin HTTPS aparțin instituției. Pentru o implementare de producție, ar fi utilă monitorizarea schimbărilor majore, validarea certificatelor, auditarea configurației de domenii și păstrarea unui istoric al indexărilor.

6.8 Direcții viitoare

Direcțiile de dezvoltare includ reindexare programată, monitorizarea modificărilor în sursele oficiale, extinderea setului de teste, evaluare cu studenți reali și îmbunătățirea OCR-ului pentru documente scanate. O altă direcție utilă este construirea unui set standard de întrebări și răspunsuri de referință, pentru a compara obiectiv variante diferite de embedding, reranking și modele generative.

La nivel de interfață, extensia poate fi extinsă cu explicații mai clare pentru întrebările neacoperite, gruparea surselor pe categorii și sugestii de reformulare. La nivel de backend, ar fi utilă separarea mai explicită între răspunsuri de navigare, răspunsuri administrative și răspunsuri de regulament, astfel încât fiecare categorie să aibă reguli de verificare adaptate.

Capitolul 7

Concluzii

Lucrarea a prezentat proiectarea unui asistent inteligent local pentru accesul la informațiile oficiale UVT. Problema abordată este una practică: informația există, dar este distribuită între mai multe site-uri și nu este întotdeauna ușor de găsit rapid. UVT Asist propune o interfață conversațională integrată în browser, susținută de un backend care caută, verifică și sintetizează informații din surse oficiale.

Din punct de vedere teoretic, soluția se bazează pe combinarea regăsirii informației cu modelele de limbaj. Căutarea semantică oferă flexibilitate, regulile deterministe oferă control, iar generarea locală oferă o interfață naturală. Prin afișarea surselor, scorului de încredere și stării de verificare, sistemul urmărește să rămână transparent și prudent.

Prototipul demonstrează că o arhitectură RAG locală este fezabilă pentru un scenariu universitar. Totuși, calitatea finală depinde de acoperirea indexului, de actualizarea surselor și de evaluarea continuă pe întrebări reale. Cele mai importante direcții de maturizare sunt automatizarea reindexării, evaluarea sistematică și îmbunătățirea mecanismelor de clarificare pentru întrebările ambigue.

Capitolul 8

Plan de activitate

Activitatea de realizare a lucrării a fost organizată în etape succesive:

- **Decembrie – Ianuarie:** documentare și analiză. Au fost studiate conceptele de bază privind asistenții conversaționali, extensiile Google Chrome, arhitectura web a UVT, retrieval-ul și modelele conversaționale.
- **Ianuarie – Februarie:** proiectare arhitecturală. Au fost definite componentele sistemului, fluxul de procesare, structura extensiei și responsabilitățile backend-ului.
- **Februarie – Martie:** implementarea primelor componente. Au fost dezvoltate extensia Chrome, endpoint-urile Flask și mecanismele inițiale de căutare și răspuns.
- **Aprilie:** integrarea cu sursele oficiale UVT. Au fost dezvoltate mecanismele de indexare, extragere a textului, segmentare, clasificare a paginilor și căutare în surse oficiale.
- **Mai – Iunie:** rafinare, testare și redactare. Au fost adăugate Qdrant, embeddings locale, reranking determinist, scor de încredere, stări de interfață, feedback și textul final al lucrării.

Bibliografie

- [Goo] Google Chrome Developers. Chrome extensions: Manifest v3 documentation. <https://developer.chrome.com/docs/extensions/>. Accesat: 26 mai 2026.
- [LPP⁺20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [MRS08] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schutze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [Oll] Ollama. Ollama documentation. <https://github.com/ollama/ollama/tree/main/docs>. Accesat: 26 mai 2026.
- [Pal] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com/>. Accesat: 26 mai 2026.
- [Qdr] Qdrant. Qdrant documentation. <https://qdrant.tech/documentation/>. Accesat: 26 mai 2026.
- [RZ09] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.